University of Exeter

ECM2433 The C Family

C++ Memory, pointers, and references

Pointers in C++

University of Exeter

C constructions (STILL VALID)

```
int *p = (int *)malloc(sizeof(int)*25);
p[4] = 43;
free(p);
```

Similarly with Classes:

```
MyClass *c;
c = (MyClass *)malloc(sizeof(MyClass));
c->data = 4;
c->memberFunction();
free(c);
```

However, the MyClass constructor and destructor are not called.

New in C++

The nullptr literal: A dedicated keyword for null pointers.

```
void f(int);    // Overload 1
void f(int*);   // Overload 2
f(NULL);     // Ambiguous: which one?
f(nullptr); // Clear: calls f(int*)
```

Type safety: C++ forbids implicit conversion from void*.

```
int *p = malloc(4); // OK in C, error in C++
int *p = (int*)malloc(4); // Cast required!
```

Pointer to class members

```
type ClassName::* pointerName;
```

```
int MyClass::*pData = &MyClass::data;
c.*pData = 42; // Access via object instance
```

Memory: new and delete

C way:

```
int *p = (int *)malloc(sizeof(int)*25);
free(p);
```

C++ way:

```
int *p = new int;        // a single integer, uninitialized
int *m = new int();      // a single integer, initialised to value 0
int *q = new int(43);    // a single integer, initialised to value 43
int *k = new int[25];    // array of 25 integers
delete p;                // free single integer
delete m;                // free single integer
delete q;                // free single integer
delete[] k;              // free array k
```

Memory: new and delete

C way:

```
MyClass *c = (MyClass *)malloc(sizeof(MyClass));
free(c);
```

```
class MyClass {
public:
    int data;
    MyClass(int val = 0){
        data = val;
    }
}
```

C++ way:

```
MyClass *b = new MyClass;       // create an object of MyClass
MyClass *c = new MyClass(42);   // create an object of MyClass, initialised with 42
MyClass *d = new MyClass[10];   // create an array of 10 MyClass objects
delete b;                       // free single object
delete c;                       // free single object
delete[] d;                     // free array d
```

Memory: new and delete

C way:

```
MyClass *c = (MyClass *)malloc(sizeof(MyClass));
free(c);
```

allocates memory
but does **not** call the constructor

frees memory
but does **not** call the destructor

C++ way:

```
MyClass *c = new MyClass;       // create an object of MyClass
MyClass *c = new MyClass(42);   // create an object of MyClass, initialised with 42
MyClass *d = new MyClass[10];   // create an array of 10 MyClass objects
delete c;                       // free single object
delete[] d;                     // free array d
```

Memory: new and delete

C way:

```
MyClass *c = (MyClass *)malloc(sizeof(MyClass));
free(c);
```

C++ way:

allocates memory
then calls the constructor

```
MyClass *c = new MyClass;      // create an object of MyClass
MyClass *c = new MyClass(42);  // create an object of MyClass, initialised with 42
MyClass *d = new MyClass[10];  // create an array of 10 MyClass objects
delete c;                      // free single object
delete[] d;                    // free array d
```

calls the destructor
then deallocates memory

References: aliases

```
#include <iostream>
int main ()
{
    int x;
    int &y = x;
    x = 42;
    std::cout << "x = " << x;
    std::cout << "  y = " << y << std::endl;

    y = 64;
    std::cout << "x = " << x;
    std::cout << "  y = " << y << std::endl;
    return 0;
}
```

A **reference**

- Provides a new name to an existing variable
- Is dereferenced implicitly (doesn't need the dereferencing operator * to obtain the value referenced)
- Cannot be null

A **pointer**

- A pointer variable stores an address
- The stored address can be changed

```
x = 42   y = 42
x = 64   y = 64
```

References: pass by value

```
void addTen(int x) //  Pass by value
{
   x += 10;
}

int main ()
{
   int num = 4;

   std::cout << "num = " << num << endl;
   addTen(num);
   std::cout << "num = " << num << endl;
   return 0;
}
```

```
num = 4
num = 4
```

References: pass by reference

```
void addTen(int &x)    // Pass by reference
{
   x += 10;
}

int main ()
{
   int num = 4;

   std::cout << "num = " << num << std::endl;
   addTen(num);
   std::cout << "num = " << num << std::endl;
   return 0;
}
```

```
num = 4
num = 14
```

References: pseudo pass by reference

University of Exeter

```
void addTen (int *x)   // Pseudo pass by reference
{                         NB: The pointer itself is passed by value!
   *x += 10;
}

int main ()
{
   int num = 4;

   std::cout << "num = " << num << std::endl;
   addTen(&num);
   std::cout << "num = " << num << std::endl;
   return 0;
}
```

```
num = 4
num = 14
```

Reference as a function return value

```
char& setValue(char *s)
{
    return s[2];
}
int main ()
{
    char str[] = "Hello World!";
    std::cout << "str = " << str << std::endl;
str[2] setValue(str) = 'X';
    std::cout << "str = " << str << std::endl;
    return 0;
}
```

The function call appears on the left hand side of an assignment operator!

```
str = Hello World!
str = HeXlo World!
```

References: pass by const reference

const: promise not to change this value

```
void analyzeData (const std::vector<int> &list)
{
    std::cout << "Analyzing " << list.size() << " items..." << std::endl;
    list.clear(); // Error! The promise is kept.
}
int main () {
    std::vector<int> hugeList(1000000);
    analyzeData(hugeList);
    return 0;
}
```

```
Analyzing 1000000 items…
```

If we passed by value (without &), C++ would copy all 1,000,000 integers. Using const & makes it fast (no copy is made) and safe (data cannot be changed)

More const

```
int val = 10;
int other = 20;

const int* p_data = &val;
*p_data = 5;                // Error! P_data is a pointer to const int
p_data = &other;            // OK! pointer can move

int* const p_ptr = &val;
*p_ptr = 5;                 // OK! Data is mutable
p_ptr = &other;             // Error! p_ptr is a const pointer


const std::vector<int> vec{ 1, 2, 3 };
std::vector<int>& ref_vec = vec;          // Error! can't declare a non-const
                                          // reference to a const variable
const std::vector<int>& ref_vec = vec;    // OK!
```

Reminder: constructors and destructors

```
#include <Stack.h>

int main ()
{
   Stack s1;
   int   value;


   s1.push(23);

   s1.push(1);

   value = s1.pop();


   return 0;
}
```

constructor called

destructor called as object goes out of scope
Also with `delete` operator:
`delete s1`

RAII

Resource Acquisition Is Initialisation. RAII binds the life cycle of a resource that must be acquired before use to the lifetime of an object.

What is the C++ object lifetime?

Objects have a defined beginning of life, and end of life. Both of those events have code which will automatically run: namely, constructors and destructors.

RAII means:

- Resource acquisition is done in the constructor of a RAII class
- Resource disposal is done in the destructor of a RAII class

RAII Compliant?

std::mutex - a class in standard library for shared resource access control

```
void func1(std::mutex& m)
{
    m.lock(); // acquire the mutex
    if (!everything_ok())
        return;
    m.unlock();

}
```

May early return, the mutex is never released

```
void safe_func1(std::mutex& m)
{
    std::lock_guard<std::mutex> lg{m}
    if (!everything_ok())
        return;
    m.unlock();    ← no longer necessary

}
```

std::lock_guard – RAII class, the wrapper for mutexes that attempts to acquire the passed in lock. It releases the lock once it goes out of scope. Read more here.

Smart pointers

RAII for locks → std::lock_guard

RAII for memory → smart pointers

Since C++11, the "Best Practice" is to **avoid calling new and delete explicitly**

Three types of RAII-compliant pointers:

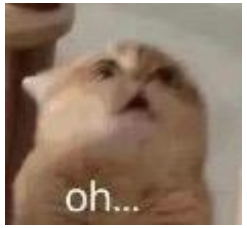
- `std::unique_ptr<T>`
  - Uniquely owns its resource, can't be copied
- `std::shared_ptr<T>`
  - Can make copies, destructed when the ***underlying memory*** goes out of scope
- `std::weak_ptr<T>`
  - A pointer to an object owned by a shared_ptr; must be converted to a shared_ptr to access the object. Read more here.

std::unique_ptr

```
struct Node {
    int data;
    Node* next;
};

void ptrFunc1() {
    Node* n = new Node;
    // do something with n
    delete n;
}

void ptrFunc2() {
    std::unique_ptr<Node> n(new Node);
    // do something with n
    // n automatically freed when it goes out of scope
}
```



std::shared_ptr

```
struct Node {
    int data;
    Node* next;
};


void ptrFunc3() {
    std::unique_ptr<Node> n(new Node);
    std::unique_ptr<Node> copy = n; //Error

    std::shared_ptr<Node> p(new Node);
    std::shared_ptr<Node> q = p;    //OK
}
```

- multiple owners
- not deallocating the underlying memory until all shared pointers go out of scope!

Summary

- C++ supports dynamic memory management using **new** and **delete**, which integrate with constructors and destructors.
- **References** provide aliases to variables and enable pass-by-reference without pointer syntax.
- **Pass by const reference** avoids copying large objects while preventing modification.
- **RAII** (Resource Acquisition Is Initialization) ties resource lifetime to object lifetime.
- **Smart pointers** (std::unique_ptr, std::shared_ptr, std::weak_ptr) provide RAII-compliant memory management.